



VACUUM

PostgreSQL

What is VACUUM?

VACUUM is a PostgreSQL command for cleaning and optimizing the database

Main Functions:

Removes "dead" rows (dead tuples)

Frees space for reuse

Updates table statistics

Doesn't return space to OS

Basic Syntax:

```
VACUUM [options] [table_name];
```

Why is VACUUM Needed?

PostgreSQL uses MVCC (Multi-Version Concurrency Control)

During UPDATE/DELETE operations, old row versions remain

These "dead" rows occupy space

Without cleanup, the database "bloats"

Problems without VACUUM:

Increased database file sizes

Slower queries

Outdated statistics for the query planner

Possible transaction ID wraparound issues

Types of VACUUM

VACUUM (regular)

Cleans dead rows

Frees space for reuse within the file

Doesn't block reads/writes

Doesn't return space to the operating system

Fast and safe

```
VACUUM table_name;
```

VACUUM FULL

Completely rebuilds the table

BLOCKS the table (exclusive lock)

Returns space to the OS

Requires additional disk space

```
VACUUM FULL table_name;
```

Main VACUUM Options

Popular Options:

VERBOSE - detailed output

ANALYZE - updates statistics for the planner

FREEZE - prevents transaction counter wraparound

FULL -complete table rebuild (blocks table)

PARALLEL - parallel execution (PostgreSQL 13+)

INDEX_CLEANUP - index cleanup

Syntax with options:

```
VACUUM (VERBOSE, ANALYZE) table_name;
```

Advanced VACUUM Options

Additional Options:

SKIP_LOCKED - skip locked tables

TRUNCATE - return empty pages to OS

PROCESS_TOAST - process TOAST tables

PROCESS_MAIN - process main table

DISABLE_PAGE_SKIPPING - scan all pages

ONLY_DATABASE_STATS - update only database stats

BUFFER_USAGE_LIMIT - limit memory usage

Transaction ID Wraparound Issue

The Problem:

PostgreSQL assigns each transaction a unique ID (XID)

XIDs are 32-bit numbers → limited to ~4.3 billion

When limit reached, IDs "wrap around" back to 0

What happens without VACUUM:

Database shutdown when approaching limit

Data becomes invisible due to ID confusion

Critical system failure

How VACUUM prevents this:

"Freezes" old transaction IDs to special permanent value

Resets the counter safely

This is why autovacuum should NEVER be disabled

Autovacuum - Automatic Cleanup

What is it?

PostgreSQL automatically runs VACUUM in background

Also runs auto-analyze (ANALYZE)

Keeps your database healthy 24/7

When it triggers:

When ~20% of table rows become "dead"

Can be customized per table

Key point: keep autovacuum enabled - it prevents most performance issues

Monitoring VACUUM

Red flags to watch:

Tables not vacuumed in weeks

Growing dead tuple counts

Sudden query slowdowns

Simple health check:

```
SELECT tablename, last_vacuum, last_autovacuum,  
       n_dead_tup, n_live_tup  
FROM pg_stat_user_tables  
WHERE schemaname = 'public';
```

Tools:

pgAdmin for GUI management

pg_stat_statements for query analysis

pg_repack for online table rebuilds

Best Practices - Do's and Don'ts

✓ DO:

Keep autovacuum enabled

Run VACUUM ANALYZE after large data changes

Monitor table health periodically

Review autovacuum settings for large tables

✗ DON'T:

Never disable autovacuum globally

Avoid VACUUM FULL on production without downtime

Don't run VACUUM FULL without 2x free disk space

🎯 **Golden Rule:** When in doubt, run `VACUUM ANALYZE table_name;`

Useful Resources & Links

Official Documentation:

[PostgreSQL VACUUM Documentation](#)

[Autovacuum Configuration](#)

[Statistics Monitoring](#)

System Tables for Monitoring:

`pg_stat_user_tables` - table statistics

`pg_stat_activity` - active processes

`pg_stat_progress_vacuum` - VACUUM progress

Thank you

- Author: Denys Kuchmei
- My LinkedIn: <https://www.linkedin.com/in/denys-kuchmei-7a8259208/>
- Date: June 2025
- [Join Codeus community in Discord](#)
- [Join Codeus community in LinkedIn](#)